



نام درس: سیستم های توزیع شده

اعضای گروه:

فرشته باقری - ۸۱۰۱۰۰۰۸۹

الهه خداوردی - ۸۱۰۱۰۰۱۳۲

محمد امانلو - ۸۱۰۱۰۰۰۸۴



تمرین شماره ۳

مقدمه

الگوریتم Raft یکی از الگوریتم های اجماع است که هدف اصلی آن Understandability یا «قابل فهم بودن» است. سادگی، کوچک بودن فضای حالت، و تجزیه پذیری الگوریتم به بخش های مجزا و ساده از دلایل محبوبیت آن هستند. در این پروژه، نسخه ی توسعه یافته ی الگوریتم Raft بر اساس مقاله ی Extended Raft پیاده سازی شده است. این پروژه شامل چهار بخش اصلی است:

□ بخش A: پیاده سازی انتخاب رهبر و استفاده از AppendEntries به عنوان heartbeat.

□ بخش B: پیاده سازی log replication و کامیت/اعمال کردن دستورات.

□ بخش C: پیاده سازی Crash Recovery و State Persistence.

□ بخش D: پیاده سازی Snapshot و Log Compaction.

در بخش های A تا D از مقاله ی اصلی الگوریتم استفاده شده و تمام متغیرها و قوانین مطابق آن هستند.

شرح گام به گام پیاده سازی پروژه

در این بخش، فرآیند کامل پیاده سازی پروژه بر اساس آنچه در صورت پروژه به وضوح خواسته شده است، به صورت گام به گام شرح داده می شود. هدف از این بخش، نمایش دقیق مسیر انجام پروژه، تطابق کامل با نیازمندی ها، و تحلیل مراحل کلیدی است.

تعاریف مهم و کلیدواژه ها

برای فهم بهتر گزارش و ساختار پروژه، ابتدا به برخی تعاریف مهم و کلیدواژه هایی که در کل پروژه به کار می روند اشاره می کنیم:

□ Log Replication: فرآیندی که در آن Leader ورودی های جدید را در Log خود ثبت کرده و آن ها را به Follower ها ارسال می کند تا با اکثریت هماهنگ شوند.

□ Commit: زمانی رخ می دهد که یک ورودی Log در اکثریت سرورها ثبت شود و آماده ی اعمال در State Machine شود.

□ Snapshot: ذخیره ی یک نسخه ی فشرده از وضعیت سیستم برای جلوگیری از رشد بی رویه ی Log.

□ Crash Recovery: توانایی سیستم برای بازیابی وضعیت پایدار پس از خاموش شدن یا از کار افتادن سرور.

□ Persistent State: متغیرهایی از وضعیت Raft که باید روی دیسک ذخیره شوند تا پس از راه اندازی مجدد حفظ شوند.

گام اول: ساختار اولیه و تابع Make

مطابق صورت پروژه، ابتدا فایل raft.go فراهم شده را بررسی کرده و ساختار اولیه ی داده ها و Interface های مورد نیاز را تحلیل کردیم. سپس، ساختار اصلی Raft را با استفاده از تعریف های خواسته شده تکمیل کردیم. این ساختار شامل متغیرهای وضعیت پایدار مانند currentTerm، votedFor و log و همچنین متغیرهای موقتی مانند commitIndex، nextIndex و state می شود.

سپس تابع Make(...) که در ابتدای پروژه فراخوانی می شود را تکمیل کردیم. این تابع مقداردهی اولیه ی نود را انجام می دهد، کانال ها را می سازد و یک Goroutine برای اجرای Ticker راه اندازی می کند تا بررسی وضعیت Leader به صورت دوره ای انجام شود.

گام دوم: پیاده سازی قسمت 3A - انتخاب رهبر

مطابق با صورت پروژه، در این مرحله باید مکانیزم کامل انتخابات پیاده سازی می شد. این شامل تعریف ساختار پیام های RequestVoteArgs و RequestVoteReply، پیاده سازی ارسال و دریافت این پیام ها از طریق RPC و منطق تبدیل نود به Candidate و سپس Leader بود.

در داخل Ticker، با بررسی زمان آخرین فعالیت و مقایسه با Election Timeout تصادفی، در صورت نیاز نود به وضعیت Candidate تغییر می کند، ترم جدید آغاز می شود و با ارسال درخواست رأی به سایر نودها، منتظر دریافت اکثریت آراء می ماند.

در صورت دریافت آرای کافی، نود به Leader تبدیل می شود و به طور مرتب پیام های AppendEntries خالی (به عنوان Heartbeat) به سایر نودها ارسال می کند.

گام سوم: پیاده سازی قسمت 3B - ثبت و اعمال Log

در این بخش از پروژه، طبق صورت تمرین، مسئولیت Leader در انتشار ورودی های جدید Log پیاده سازی شد. تابع `Start(command)` توسط لایه ی بالایی برای ارسال دستور فراخوانی می شود. Leader ورودی را به Log خود اضافه کرده و از طریق پیام های `AppendEntries` آن را به Follower ها ارسال می کند. پس از اطمینان از ثبت شدن ورودی در اکثریت نودها، آن را `Commit` می کند و از طریق کانال `applyCh` برای اجرا ارسال می شود.

Follower ها نیز پیام های `AppendEntries` را دریافت می کنند، صحت `PrevLogIndex` و `PrevLogTerm` را بررسی کرده و در صورت تطابق، ورودی های جدید را اضافه می کنند. مکانیزم بازگشتی در صورت ناسازگاری Log نیز مطابق با نسخه ی توسعه یافته ی الگوریتم پیاده سازی شد.

گام چهارم: پیاده سازی قسمت 3C - پایداری و بازیابی

بر اساس صورت پروژه، باید وضعیت پایدار نودها از طریق ساختار `Persister` ذخیره می شد. در این مرحله، توابع `persist()`، `saveStateAndSnapshot()` و `readPersist()` پیاده سازی شدند. در هر بار تغییر وضعیت پایدار (مثل رأی دهی یا افزودن ورودی جدید)، وضعیت فعلی رمزنگاری شده و با استفاده از `Persister.Save()` ذخیره می شود. همچنین هنگام شروع مجدد سرور، تابع `readPersist()` فراخوانی می شود تا این وضعیت از دیسک بازیابی شده و نود بتواند از همان وضعیت قبلی به کار ادامه دهد.

گام پنجم: پیاده سازی قسمت 3D - فشرده سازی Log و Snapshot

در این مرحله، مطابق صورت پروژه، تابع `Snapshot(index, snapshot)` پیاده سازی شد. این تابع توسط لایه ی بالا فراخوانی می شود و هدف آن حذف ورودی های قدیمی Log و حفظ فقط وضعیت فشرده شده ی سیستم است. ابتدا بررسی می شود که ایندکس داده شده منطقی و جلوتر از آخرین `Snapshot` باشد. سپس Log قدیمی بریده می شود و یک ورودی جعلی در ابتدای Log باقی مانده قرار می گیرد تا پیوستگی حفظ شود. مقادیر `lastIncludedIndex` و `lastIncludedTerm` به روزرسانی می شوند و سپس وضعیت جدید همراه با `Snapshot` ذخیره می شود.

در صورتی که نود در حالت Leader باشد، مقادیر `nextIndex` و `matchIndex` برای همه Follower ها نیز به نحوی اصلاح می شوند که از ارسال ورودی های حذف شده جلوگیری شود.

همچنین تابع `InstallSnapshot` برای ارسال `Snapshot` از Leader به Follower هایی که Log آن ها عقب تر از `Snapshot` است، پیاده سازی شد. Follower پس از دریافت `Snapshot`، Log قدیمی را حذف کرده و فقط وضعیت جدید را نگه می دارد.

با اجرای تست‌های موجود در فایل raft test.go اطمینان حاصل شد که پیاده‌سازی انجام‌شده تمامی سناریوهای مهم مانند Split Vote Election Timeout، Log Inconsistency، Crash Recovery، و Snapshot Recovery را با موفقیت پشت سر می‌گذارد.

ساختارهای داده‌ای

در این بخش، ساختارهای پایه‌ای مورد استفاده در پیاده‌سازی الگوریتم Raft معرفی می‌شوند. این ساختارها پایه‌های ارتباطی بین سرورها، نگهداری وضعیت هر نود، و مدیریت لاگ‌ها را تشکیل می‌دهند.

وضعیت نودها (حالت سرور)

هر سرور در الگوریتم Raft در هر لحظه تنها می‌تواند در یکی از سه وضعیت زیر قرار داشته باشد:

```
1 const (  
2     follower = 0  
3     candidate = 1  
4     leader = 2  
5 )
```

□ follower: حالت پیش‌فرض نود که تنها به پیام‌های رهبر یا کاندیدها پاسخ می‌دهد.

□ candidate: زمانی که تایم‌اوت دریافت heartbeat رخ دهد، سرور به کاندید تبدیل می‌شود و تلاش می‌کند رأی‌گیری را شروع کند.

□ leader: نودی که موفق شده رأی اکثریت را به دست آورد و مسئول هماهنگی و ارسال لاگ به دیگر نودها است.

ساختار ورودی‌های لاگ (LogEntry)

این ساختار برای نمایش هر ورودی در لاگ Raft استفاده می‌شود:

```
1 type LogEntry struct {  
2     Term      int  
3     Command interface{}  
4 }
```

□ Term: ترمی که این دستور در آن اضافه شده است. این عدد برای مقایسه‌ی به‌روز بودن لاگ در زمان رأی‌گیری و هماهنگی‌ها استفاده می‌شود.

□ Command: دستور واقعی که از طرف کلاینت دریافت شده و در صورت کامیت شدن باید توسط ماشین وضعیت اجرا شود. نوع آن interface است تا انواع مختلف داده‌ها را پشتیبانی کند.

ساختارهای مربوط به RPC درخواست رأی (RequestVote)

Raft از فراخوانی‌های از راه دور (RPC) برای انتخابات استفاده می‌کند. ساختار زیر ورودی و خروجی این پیام را تعریف می‌کند:

```
1 type RequestVoteArgs struct {  
2     Term          int  
3     CandidateId   int  
4     LastLogIndex  int  
5     LastLogTerm   int  
6 }  
7  
8 type RequestVoteReply struct {  
9     Term          int  
10    VoteGranted    bool  
11 }
```

توضیح فیلدها:

□ Term: ترم فعلی کاندیدا که برای اطلاع‌رسانی به فالوئر‌ها ارسال می‌شود.

□ CandidateId: شناسه عددی نودی که قصد دارد رهبر شود.

□ LastLogIndex, LastLogTerm: مشخص می‌کنند که لاگ نود کاندید تا چه اندازه به‌روز است؛ فالوئر فقط در صورتی رأی می‌دهد که لاگ کاندید حداقل به اندازه خودش جدید باشد.

□ VoteGranted: فیلدی در پاسخ که نشان می‌دهد آیا رأی داده شده یا خیر.

ساختار پیام‌های الحاق لاگ (AppendEntries)

این ساختار برای ارسال لاگ‌ها یا heartbeat از طرف رهبر به فالوئر‌ها استفاده می‌شود.

```
1 type AppendEntriesArgs struct {  
2     Term          int  
3     LeaderId      int  
4     PrevLogIndex  int  
5     PrevLogTerm   int  
6     Entries       [] LogEntry  
7     LeaderCommit  int  
8 }  
9  
10 type AppendEntriesReply struct {  
11     Term          int  
12     Success       bool  
13     ConflictIndex int  
14     ConflictTerm  int  
15 }
```

کاربرد فیلدها:

□ Term، LeaderId: مشخص‌کننده اطلاعات رهبر فعلی هستند.

□ PrevLogIndex، PrevLogTerm: برای اطمینان از اتصال درست لاگ، رهبر باید ورودی قبلی را نیز مشخص کند.

□ Entries: ورودی‌های جدید لاگ که باید اضافه شوند.

□ LeaderCommit: مقدار commitIndex رهبر که فالوئر‌ها باید با آن هماهنگ شوند.

□ ConflictIndex، ConflictTerm: اگر لاگ تطابق نداشته باشد، این فیلدها به رهبر کمک می‌کنند که سریع‌تر به محل صحیح لاگ برسد (در نسخه extended).

ساختار مربوط به Snapshot

برای فشرده سازی لاگ، از Snapshot استفاده می شود تا از ذخیره ورودی های قدیمی جلوگیری شود.

```

1 type InstallSnapshotArgs struct {
2     Term          int
3     LeaderId      int
4     LastIncludedIndex int
5     LastIncludedTerm int
6     Data          []byte
7 }
8
9 type InstallSnapshotReply struct{
10     Term int
11 }

```

در این ساختار:

□ LastIncludedIndex, LastIncludedTerm: مشخص می کنند Snapshot شامل چه بخشی از لاگ است.

□ Data: بایت های فشرده شده ی Snapshot که باید روی فالوئر جایگزین شود.

ساختار اصلی نود Raft

ساختار Raft قلب الگوریتم است و تمام اطلاعات مربوط به وضعیت نود، لاگ، رای دهی، وضعیت رهبر یا فالوئر بودن را نگه می دارد.

```

1 type Raft struct {
2     mu          sync.Mutex
3     peers       []*labrpc.ClientEnd
4     persister   *tester.Persister
5     me          int
6     dead        int32
7
8     currentTerm int

```

```

9      votedFor          int
10     log                [] LogEntry
11     lastIncludedIndex int
12     lastIncludedTerm  int
13
14     commitIndex int
15     lastApplied int
16
17     nextIndex [] int
18     matchIndex [] int
19
20     state          int
21     applyCh        chan raftapi.ApplyMsg
22     lastResetOfTicker time.Time
23 }

```

توضیح اجزای مهم:

- mu: برای همزمانی بین رشته‌های اجرایی (goroutine) و جلوگیری از شرایط رقابتی.
- peers: لیستی از نودهای دیگر در کلاستر.
- persister: برای نگهداری وضعیت در دیسک (در برابر crash).
- log: لیست دستورات ثبت شده.
- nextIndex و matchIndex: فقط در رهبر نگه داشته می‌شوند؛ برای ارسال لاگ‌ها به فالوئر‌ها.
- commitIndex: آخرین اندیسی که می‌دانیم به صورت امن روی اکثریت نودها ثبت شده است.
- applyCh: کانالی برای ارسال دستورات کامیت شده به لایه بالاتر (State Machine).

توابع کمکی

الگوریتم Raft نیازمند مجموعه‌ای از توابع ساده اما بسیار حیاتی است که در سراسر پیاده‌سازی استفاده می‌شوند. این توابع برای جلوگیری از تکرار کد، افزایش خوانایی و تسهیل در مدیریت اندیس‌ها و زمان‌بندی‌ها طراحی شده‌اند. در ادامه، این توابع همراه با کاربرد دقیق و اهمیت آن‌ها بررسی می‌شوند:

تبدیل ایندکس جهانی به ایندکس محلی لاگ

```
1 func (rf *Raft) slice(i int) int {
2     return i - rf.lastIncludedIndex
3 }
```

این تابع برای نگاشت یک ایندکس جهانی (مثلاً `commitIndex`) به ایندکس داخلی آرایه‌ی `log` مورد استفاده قرار می‌گیرد. به دلیل استفاده از `Snapshot`، ممکن است لاگ شامل تمام ورودی‌های اولیه نباشد؛ بنابراین باید بدانیم ورودی با ایندکس `i` در چه موقعیتی از `log` فعلی قرار دارد. اگر `lastIncludedIndex` برابر با `۱۰` باشد، یعنی لاگ ورودی‌های `۱` تا `۱۰` را حذف کرده و ایندکس `۱۱` در موقعیت `۰` از آرایه قرار دارد.

دریافت آخرین ایندکس لاگ

```
1 func (rf *Raft) lastLogIndex() int {
2     return rf.lastIncludedIndex + len(rf.log) - 1
3 }
```

این تابع ایندکس آخرین ورودی موجود در لاگ را برمی‌گرداند. به دلیل وجود `Snapshot`، آرایه‌ی `log` ممکن است از وسط شروع شود. در این صورت، ایندکس آخر باید با اضافه کردن طول فعلی `log` به `lastIncludedIndex` محاسبه شود.

دریافت ترم آخرین ورودی لاگ

```
1 func (rf *Raft) lastLogTerm() int {
2     return rf.log[len(rf.log) - 1].Term
3 }
```

این تابع برای به دست آوردن شماره ی ترم مربوط به آخرین ورودی لاگ استفاده می شود. این مقدار هنگام مقایسه ی لاگ ها در رأی گیری یا بررسی به روز بودن فالوئر ها بسیار کاربردی است. فرض بر این است که همیشه حداقل یک ورودی در log وجود دارد (ورودی جعلی در ابتدای لاگ پس از Snapshot).

گرفتن ترم مربوط به یک ایندکس خاص در لاگ

```
1 func (rf *Raft) termAt(i int) int {
2     if i == rf.lastIncludedIndex {
3         return rf.lastIncludedTerm
4     }
5     return rf.log[rf.slice(i)].Term
6 }
```

زمانی که نیاز داریم ترم مربوط به ورودی شماره i در لاگ را بیابیم (مثلاً هنگام بررسی مطابقت لاگ)، از این تابع استفاده می کنیم. نکته ی مهم این است که اگر i برابر با lastIncludedIndex باشد، این ورودی دیگر در لاگ موجود نیست و تنها نماینده ی آن در فیلد lastIncludedTerm ذخیره شده است. در غیر این صورت، باید از تابع slice برای یافتن مکان دقیق در آرایه ی فعلی log استفاده کرد.

محاسبه ی تصادفی زمان پایان تایم اوت انتخابات

```
1 func (rf *Raft) electionTimeout() time.Duration {
2     return time.Duration(300+rand.Intn(200)) * time.Millisecond
3 }
```

این تابع مقدار تصادفی بین ۳۰۰ تا ۵۰۰ میلی ثانیه تولید می کند که برای تایم اوت انتخابات استفاده می شود. دلیل تصادفی بودن آن، جلوگیری از همزمان شدن انتخابات در چند نود است که باعث split vote می شود. این یکی از مکانیزم های کلیدی Raft برای رسیدن سریع تر به اجماع است.

فاصله ی بین ارسال heartbeat توسط رهبر

```
1 func (rf *Raft) heartbeatInterval() time.Duration {
2     return 100 * time.Millisecond
3 }
```

هر رهبر باید در بازه های زمانی مشخص، پیام AppendEntries خالی به فالوئر ها ارسال کند تا نشان دهد که هنوز زنده است. این تابع این فاصله ی زمانی را تعیین می کند. اگر فالوئر برای مدتی مشخص چنین پیامی دریافت نکند، فرض می کند رهبر از دست رفته و وارد انتخابات می شود.

بازنشانی تایمر انتخابات

```
1 func (rf *Raft) resetTimer() {
2     rf.lastResetOfTicker = time.Now()
3 }
```

هر بار که نود پیامی از رهبر دریافت می کند (مثلاً AppendEntries) یا رأی می دهد، تایمر انتخابات باید بازنشانی شود. این تابع زمان آخرین فعالیت مهم نود را ثبت می کند تا در بررسی بعدی بدانیم آیا زمان کافی برای شروع انتخابات گذشته یا خیر.

بررسی وضعیت نود از بیرون

```
1 func (rf *Raft) GetState() (int, bool) {
2     rf.mu.Lock()
3     defer rf.mu.Unlock()
4     return rf.currentTerm, rf.state == leader
5 }
```

این تابع معمولاً از سمت لایه ی بیرونی (مثلاً کلاینت یا تست) برای بررسی وضعیت نود استفاده می شود. دو مقدار برمی گرداند:

□ currentTerm: ترم فعلی که نود در آن قرار دارد.

□ isLeader: یک مقدار bool که مشخص می کند آیا نود در حال حاضر رهبر است یا خیر.

با استفاده از این تابع می توان وضعیت جاری سیستم را در حین اجرا مشاهده و ارزیابی کرد.

توابع اصلی

در این بخش، به بررسی توابع کلیدی و اصلی الگوریتم Raft می پردازیم. این توابع عمدتاً برای پشتیبانی از ویژگی های مهمی مانند Crash Recovery، پایداری وضعیت (Persistent State)، و همچنین فشرده سازی لاگ از طریق Snapshot

به کار می‌روند.

الگوریتم Raft به‌طور خاص برای تحمل خرابی طراحی شده است، بنابراین بخش مهمی از عملکرد آن به حفظ وضعیت پایدار بین اجرای مجدد گره‌ها مربوط می‌شود. به همین دلیل، ما از ساختار `persister` برای نوشتن وضعیت در حافظه‌ی دائمی (مانند دیسک) استفاده می‌کنیم. همچنین برای محدود کردن رشد بی‌رویه‌ی لاگ، مکانیزمی به نام `Snapshotting` در این پروژه پیاده‌سازی شده است.

ذخیره وضعیت و Snapshot

```

1 func (rf *Raft) saveStateAndSnapshot(snap []byte) {
2     w := new(bytes.Buffer)
3     e := labgob.NewEncoder(w)
4     e.Encode(rf.currentTerm)
5     e.Encode(rf.votedFor)
6     e.Encode(rf.log)
7     e.Encode(rf.lastIncludedIndex)
8     e.Encode(rf.lastIncludedTerm)
9     rf.persister.Save(w.Bytes(), snap)
10 }
11
12 func (rf *Raft) persist() {
13     rf.saveStateAndSnapshot(rf.persister.ReadSnapshot())
14 }

```

تابع `saveStateAndSnapshot` مسئول ذخیره‌سازی کامل وضعیت نود در حافظه‌ی پایدار است. عملکرد آن به این صورت است که ابتدا با استفاده از `bytes.Buffer` یک حافظه‌ی موقتی در رم ایجاد می‌کند. سپس به کمک `labgob.NewEncoder` داده‌های مهم وضعیت را سریال‌سازی کرده و در این حافظه موقتی می‌نویسد. این داده‌ها شامل موارد زیر هستند:

□ `currentTerm`: آخرین ترمی که نود در آن قرار داشته است.

□ `votedFor`: شماره شناسه‌ی نودی که در این ترم به آن رأی داده شده (اگر هیچ‌کدام نباشد مقدار -1 است).

□ `log`: کل ورودی‌های باقی‌مانده در لاگ بعد از `Snapshot`.

□ lastIncludedIndex و lastIncludedTerm: نماینده‌ی بخش فشرده‌شده‌ی لاگ در Snapshot.

در نهایت، داده‌های سریال شده به همراه snapshot دریافتی به تابع Save از ساختار persister تحویل داده می‌شود که مسئول ذخیره‌سازی دائمی در حافظه‌ی غیر فرار است.

تابع persist یک تابع ساده‌ی میان‌رو (wrapper) است که وضعیت را با همان Snapshot قبلی ذخیره می‌کند. این تابع در مواقعی که فقط وضعیت تغییر کرده اما Snapshot جدیدی تولید نشده، مفید است.

خواندن وضعیت ذخیره‌شده

```
1 func (rf *Raft) readPersist() {
2     data := rf.persister.ReadRaftState()
3     if len(data) > 0 {
4         r := bytes.NewBuffer(data)
5         d := labgob.NewDecoder(r)
6
7         var ct, vf int
8         var lg []LogEntry
9         var li, lt int
10
11         if d.Decode(&ct) != nil ||
12            d.Decode(&vf) != nil ||
13            d.Decode(&lg) != nil ||
14            d.Decode(&li) != nil ||
15            d.Decode(&lt) != nil {
16             return
17         }
18
19         rf.currentTerm      = ct
20         rf.votedFor        = vf
21         rf.log              = lg
22         rf.lastIncludedIndex = li
23         rf.lastIncludedTerm = lt
```

```

24
25     rf.commitIndex = rf.lastIncludedIndex
26     rf.lastApplied = rf.lastIncludedIndex
27 }
28
29 if snap := rf.persister.ReadSnapshot(); len(snap) > 0 {
30     idx, term := rf.lastIncludedIndex, rf.lastIncludedTerm
31     dataCopy := make([]byte, len(snap))
32     copy(dataCopy, snap)
33     go func() {
34         rf.applyCh <- raftapi.ApplyMsg{
35             SnapshotValid: true,
36             Snapshot:      dataCopy,
37             SnapshotIndex: idx,
38             SnapshotTerm:  term,
39         }
40     }()
41 }
42 }

```

تابع `readPersist` برای بازیابی وضعیت نود پس از راه اندازی مجدد یا Crash Recovery استفاده می شود. این تابع ابتدا داده های باینری ذخیره شده را از `persister.ReadRaftState` دریافت می کند. اگر این داده ها خالی نبودند، آن ها را با کمک `labgob.NewDecoder` رمزگشایی می کند و به متغیرهای مناسب اختصاص می دهد:

`currentTerm ← ct` □

`votedFor ← vf` □

`log ← lg` □

`lastIncludedIndex ← li` □

`lastIncludedTerm ← lt` □

سپس وضعیت نود را با این مقادیر بازسازی می کند و همچنین فرض می گیرد که تا `lastIncludedIndex` تمام ورودی های لاگ اعمال شده اند؛ بنابراین هر دو مقدار `commitIndex` و `lastApplied` برابر با `lastIncludedIndex` تنظیم می شوند.

در ادامه، اگر داده ای در `Snapshot` موجود باشد، آن را نیز از `persister` می خواند، کپی می کند، و در قالب یک پیام `ApplyMsg` از طریق کانال `applyCh` برای لایه بالاتر ارسال می کند تا وضعیت ماشین وضعیت (`State Machine`) بازیابی شود. برای جلوگیری از بلاک شدن، این ارسال در یک `goroutine` انجام می شود.

ایجاد Snapshot جدید

```

1 func (rf *Raft) Snapshot(index int, snapshot []byte) {
2     rf.mu.Lock()
3     defer rf.mu.Unlock()
4
5     if index <= rf.lastIncludedIndex || index > rf.lastLogIndex() {
6         return
7     }
8
9     term := rf.termAt(index)
10    cut := index - rf.lastIncludedIndex
11    rf.log = append([]LogEntry{{Term: term}}, rf.log[cut+1:]...)
12
13    rf.lastIncludedIndex, rf.lastIncludedTerm = index, term
14    if rf.commitIndex < index {
15        rf.commitIndex = index
16    }
17    if rf.lastApplied < index {
18        rf.lastApplied = index
19    }
20
21    rf.saveStateAndSnapshot(snapshot)
22

```

```

23  if rf.state == leader {
24      last := rf.lastLogIndex()
25      for i := range rf.peers {
26          if rf.nextIndex[i] <= rf.lastIncludedIndex {
27              rf.nextIndex[i] = rf.lastIncludedIndex
28          }
29          if rf.nextIndex[i] > last+1 {
30              rf.nextIndex[i] = last + 1
31          }
32          if rf.matchIndex[i] < rf.lastIncludedIndex {
33              rf.matchIndex[i] = rf.lastIncludedIndex
34          }
35          if rf.matchIndex[i] > last {
36              rf.matchIndex[i] = last
37          }
38      }
39  }
40  }

```

این تابع توسط لایه‌ی بالایی مانند State Machine یا Service Layer صدا زده می‌شود و هدف آن کاهش حجم لاگ با تولید Snapshot است. این عملیات برای بهینه‌سازی عملکرد سیستم، کاهش مصرف حافظه و افزایش سرعت بازیابی بسیار مهم است. مراحل عملکرد این تابع به شرح زیر است:

۱. ابتدا با گرفتن lock از اجرای موازی جلوگیری می‌شود.
۲. بررسی می‌شود که ایندکس داده‌شده قبلاً Snapshot نشده باشد یا از آخرین لاگ جلو نزده باشد.
۳. ترم مربوط به آن ایندکس با تابع termAt استخراج می‌شود.
۴. لاگ از آن ایندکس به بعد نگه‌داشته می‌شود و ورودی‌های قدیمی حذف می‌شوند. برای حفظ سازگاری، یک ورودی جعلی (dummy) با همان ترم در ابتدای log جدید افزوده می‌شود.

۵. مقادیر `lastIncludedIndex` و `lastIncludedTerm` تنظیم می شوند تا Snapshot نماینده ی دقیق آن قسمت باشد.

۶. در صورت لزوم، `commitIndex` و `lastApplied` نیز به آن ایندکس افزایش داده می شوند تا نشان دهد که این ورودی ها اعمال شده اند.

۷. سپس وضعیت به همراه Snapshot جدید با فراخوانی `saveStateAndSnapshot` ذخیره می شود.

در صورتی که این نود در وضعیت leader باشد، باید شاخص های مربوط به `nextIndex` و `matchIndex` تمام فالوئر ها نیز با وضعیت جدید Snapshot هم راستا شود:

□ اگر `nextIndex[i]` کوچک تر از `lastIncludedIndex` باشد، به مقدار `lastIncludedIndex` تنظیم می شود.

□ اگر `nextIndex[i]` بزرگ تر از انتهای لاگ باشد، کاهش می یابد تا خارج از محدوده نشود.

□ همین اصلاحات برای `matchIndex[i]` نیز انجام می شود.

این به رهبر کمک می کند تا فرآیند ارسال لاگ به فالوئر ها بدون ارسال ورودی هایی که حذف شده اند، ادامه یابد.

نتایج تست ها و بررسی صحت عملکرد

در انتهای پیاده سازی پروژه و به منظور اطمینان از صحت عملکرد الگوریتم Raft توسعه یافته، تمامی تست هایی که در فایل `test.go` برای بخش های مختلف پروژه (3A تا 3D) تعریف شده اند اجرا شدند. این تست ها هر یک برای بررسی ویژگی های خاصی از الگوریتم طراحی شده اند و موفقیت در آن ها نشان دهنده پیاده سازی صحیح، پایدار و قابل اعتماد سیستم توزیع شده مبتنی بر Raft است.

در ادامه، تصاویر خروجی اجرای این تست ها آورده شده و هر بخش شرح داده شده است.

بخش 3A: رهبر شدن و انتخابات

در این بخش تست هایی برای بررسی عملکرد صحیح فرآیند انتخابات طراحی شده اند. از جمله موارد زیر:

□ `initial election (reliable network)`: بررسی انتخاب اولیه ی رهبر در شرایطی که همه نود ها در شبکه حضور دارند.

□ `election after network failure`: بررسی صحت انتخابات مجدد پس از قطع ارتباط رهبر فعلی.

multiple elections: بررسی چندین انتخابات متوالی در یک شبکه‌ی قابل اعتماد. □

نتیجه تست‌ها در تصویر زیر مشخص است. تمام تست‌ها با موفقیت اجرا شده‌اند:

```
fereshte@asus:~/Distributed-Systems-S04/Project 3/6.5840/src/raft1$ go test -run 3A
Test (3A): initial election (reliable network)...
... Passed -- time 3.1s #peers 3 #RPCs 32 #Ops 0
Test (3A): election after network failure (reliable network)...
... Passed -- time 5.1s #peers 3 #RPCs 72 #Ops 0
Test (3A): multiple elections (reliable network)...
... Passed -- time 5.5s #peers 7 #RPCs 354 #Ops 0
PASS
ok      6.5840/raft1    13.651s
fereshte@asus:~/Distributed-Systems-S04/Project 3/6.5840/src/raft1$ go test -run 3A -race
Test (3A): initial election (reliable network)...
... Passed -- time 3.0s #peers 3 #RPCs 22 #Ops 0
Test (3A): election after network failure (reliable network)...
... Passed -- time 4.6s #peers 3 #RPCs 70 #Ops 0
Test (3A): multiple elections (reliable network)...
... Passed -- time 5.5s #peers 7 #RPCs 378 #Ops 0
PASS
ok      6.5840/raft1    14.101s
```

```
fereshte@asus:~/Distributed-Systems-S04/Project 3/6.5840/src/raft1$ go test -run 3B
Test (3B): basic agreement (reliable network)...
... Passed -- time 1.1s #peers 3 #RPCs 16 #Ops 0
Test (3B): RPC byte count (reliable network)...
... Passed -- time 3.1s #peers 3 #RPCs 48 #Ops 0
Test (3B): test progressive failure of followers (reliable network)...
... Passed -- time 4.7s #peers 3 #RPCs 72 #Ops 0
Test (3B): test failure of leaders (reliable network)...
... Passed -- time 5.6s #peers 3 #RPCs 104 #Ops 0
Test (3B): agreement after follower reconnects (reliable network)...
... Passed -- time 6.1s #peers 3 #RPCs 71 #Ops 0
Test (3B): no agreement if too many followers disconnect (reliable network)...
... Passed -- time 3.8s #peers 5 #RPCs 144 #Ops 0
Test (3B): concurrent Start()s (reliable network)...
... Passed -- time 0.7s #peers 3 #RPCs 18 #Ops 0
Test (3B): rejoin of partitioned leader (reliable network)...
... Passed -- time 4.6s #peers 3 #RPCs 79 #Ops 0
Test (3B): leader backs up quickly over incorrect follower logs (reliable network)...
... Passed -- time 31.3s #peers 5 #RPCs 2046 #Ops 0
Test (3B): RPC counts aren't too high (reliable network)...
... Passed -- time 2.5s #peers 3 #RPCs 44 #Ops 0
PASS
ok      6.5840/raft1    63.589s
```

بخش 3B: تکرار لاگ و تضمین صحت آن

در این بخش، تست‌ها روی فرآیند log replication متمرکز هستند. مهم‌ترین تست‌ها در این بخش:

basic agreement: بررسی توافق نودها روی یک ورودی ساده. □

failure of leaders و progressive failure of followers: بررسی پایداری سیستم در شرایط خرابی رهبر یا فالوئرهای. □

□ concurrent Start(): بررسی اجرای همزمان درخواست‌ها.

□ leader backs up quickly: بررسی سرعت رهبر در همگام‌سازی لاگ‌های ناسازگار.

تصاویر زیر خروجی اجرای موفق بیشتر تست‌های B و همچنین یک تست ناموفق در حالت race را نشان می‌دهند:

```
fereshte@asus:~/Distributed-Systems-S04/Project 3/6.5840/src/raft1$ go test -run 3B -race
Test (3B): basic agreement (reliable network)...
... Passed -- time 1.4s #peers 3 #RPCs 16 #Ops 0
Test (3B): RPC byte count (reliable network)...
... Passed -- time 3.3s #peers 3 #RPCs 48 #Ops 0
Test (3B): test progressive failure of followers (reliable network)...
... Passed -- time 5.0s #peers 3 #RPCs 68 #Ops 0
Test (3B): test failure of leaders (reliable network)...
... Passed -- time 5.3s #peers 3 #RPCs 96 #Ops 0
Test (3B): agreement after follower reconnects (reliable network)...
Fatal: one(101) failed to reach agreement
/home/fereshte/Distributed-Systems-S04/Project 3/6.5840/src/raft1/test.go:274
/home/fereshte/Distributed-Systems-S04/Project 3/6.5840/src/raft1/raft_test.go:280
info: wrote visualization to /tmp/porcupine-505046643.html
--- FAIL: TestFailAgree3B (2.47s)
Test (3B): no agreement if too many followers disconnect (reliable network)...
... Passed -- time 3.8s #peers 5 #RPCs 128 #Ops 0
Test (3B): concurrent Start(s) (reliable network)...
... Passed -- time 0.9s #peers 3 #RPCs 18 #Ops 0
Test (3B): rejoin of partitioned leader (reliable network)...
... Passed -- time 4.4s #peers 3 #RPCs 81 #Ops 0
Test (3B): leader backs up quickly over incorrect follower logs (reliable network)...
... Passed -- time 32.7s #peers 5 #RPCs 2046 #Ops 0
Test (3B): RPC counts aren't too high (reliable network)...
... Passed -- time 2.0s #peers 3 #RPCs 40 #Ops 0
FAIL
exit status 1
FAIL 6.5840/raft1 61.388s

fereshte@asus:~/Distributed-Systems-S04/Project 3/6.5840/src/raft1$ go test -run 3C -race
Test (3C): basic persistence (reliable network)...
... Passed -- time 5.1s #peers 3 #RPCs 58 #Ops 0
Test (3C): more persistence (reliable network)...
... Passed -- time 16.1s #peers 5 #RPCs 294 #Ops 0
Test (3C): partitioned leader and one follower crash, leader restarts (reliable network)...
... Passed -- time 3.3s #peers 3 #RPCs 35 #Ops 0
Test (3C): Figure 8 (reliable network)...
... Passed -- time 41.5s #peers 5 #RPCs 1027 #Ops 0
Test (3C): unreliable agreement (unreliable network)...
... Passed -- time 2.1s #peers 5 #RPCs 1064 #Ops 0
Test (3C): Figure 8 (unreliable) (unreliable network)...
... Passed -- time 42.8s #peers 5 #RPCs 10030 #Ops 0
Test (3C): churn (reliable network)...
... Passed -- time 16.5s #peers 5 #RPCs 4057 #Ops 0
Test (3C): unreliable churn (unreliable network)...
... Passed -- time 16.3s #peers 5 #RPCs 1870 #Ops 0
PASS
ok 6.5840/raft1 144.660s
```

نکته: در اجرای تست 3B-race، یکی از تست‌ها در حالت رقابتی (race condition) با شکست مواجه شد که دلیل آن ممکن است تاخیر در همگام‌سازی نودها یا یک شرط رقابتی گذرا باشد. اما در اجرای غیر رقابتی، همه تست‌ها با موفقیت کامل عبور کرده‌اند.

بخش 3C: پایداری وضعیت و تحمل خرابی

این بخش شامل تست‌هایی است که وضعیت پایداری (Persistence) و بازیابی از خرابی (Crash Recovery) را ارزیابی می‌کند. تست‌های مهم این بخش عبارتند از:

□ basic persistence: بررسی ذخیره وضعیت در دیسک.

□ more persistence و partitioned leader + restart: شبیه‌سازی خرابی رهبر و بازیابی.

□ unreliable: تست شرایط مختلف شبکه و چرخش نودها در شرایط غیرقابل پیش‌بینی.

نتایج در تصاویر زیر به خوبی نشان می‌دهند که سیستم ما توانسته تمامی این سناریوها را به درستی مدیریت کند:

```
fereshte@asus:~/Distributed-Systems-S04/Project 3/6.5840/src/raft1$ go test -run 3C
Test (3C): basic persistence (reliable network)...
... Passed -- time 4.6s #peers 3 #RPCs 54 #Ops 0
Test (3C): more persistence (reliable network)...
... Passed -- time 17.2s #peers 5 #RPCs 314 #Ops 0
Test (3C): partitioned leader and one follower crash, leader restarts (reliable network)...
... Passed -- time 3.3s #peers 3 #RPCs 33 #Ops 0
Test (3C): Figure 8 (reliable network)...
... Passed -- time 28.7s #peers 5 #RPCs 739 #Ops 0
Test (3C): unreliable agreement (unreliable network)...
... Passed -- time 1.9s #peers 5 #RPCs 1079 #Ops 0
Test (3C): Figure 8 (unreliable) (unreliable network)...
... Passed -- time 32.3s #peers 5 #RPCs 9355 #Ops 0
Test (3C): churn (reliable network)...
... Passed -- time 16.2s #peers 5 #RPCs 8334 #Ops 0
Test (3C): unreliable churn (unreliable network)...
... Passed -- time 16.2s #peers 5 #RPCs 3037 #Ops 0
PASS
ok      6.5840/raft1    120.328s

fereshte@asus:~/Distributed-Systems-S04/Project 3/6.5840/src/raft1$ go test -run 3D -race
Test (3D): snapshots basic (reliable network)...
... Passed -- time 4.4s #peers 3 #RPCs 488 #Ops 0
Test (3D): install snapshots (disconnect) (reliable network)...
... Passed -- time 50.6s #peers 3 #RPCs 1167 #Ops 0
Test (3D): install snapshots (disconnect) (unreliable network)...
... Passed -- time 84.7s #peers 3 #RPCs 1636 #Ops 0
Test (3D): install snapshots (crash) (reliable network)...
... Passed -- time 34.1s #peers 3 #RPCs 992 #Ops 0
Test (3D): install snapshots (crash) (unreliable network)...
... Passed -- time 49.7s #peers 3 #RPCs 1072 #Ops 0
Test (3D): crash and restart all servers (unreliable network)...
... Passed -- time 26.0s #peers 3 #RPCs 377 #Ops 0
Test (3D): snapshot initialization after crash (unreliable network)...
... Passed -- time 5.2s #peers 3 #RPCs 72 #Ops 0
PASS
ok      6.5840/raft1    255.718s
```

بخش 3D: فشرده‌سازی لاگ و Snapshot

در این بخش، مکانیزم Snapshotting بررسی می‌شود که برای جلوگیری از رشد بیش از حد لاگ استفاده می‌شود. از جمله تست‌ها:

□ install snapshot و basic snapshot: بررسی ارسال و نصب صحیح Snapshot ها.

□ crash + snapshot initialization: شبیه سازی بازیابی پس از خرابی با استفاده از Snapshot.

نتایج زیر نشان می دهند که تمام تست های این بخش نیز با موفقیت به اتمام رسیده اند:

```

fereshtegasus@~/Distributed-Systems-504/Project 3/6.5840/src/raft1$ go test -race
Test (3A): initial election (reliable network)...
warning: term changed even though there were no failures ... Passed -- time 3.1s #peers 3 #RPCs 34 #Ops 0
Test (3A): election after network failure (reliable network)...
... Passed -- time 4.6s #peers 3 #RPCs 62 #Ops 0
Test (3A): multiple elections (reliable network)...
... Passed -- time 6.7s #peers 7 #RPCs 378 #Ops 0
Test (3B): basic agreement (reliable network)...
... Passed -- time 1.3s #peers 3 #RPCs 16 #Ops 0
Test (3B): RPC byte count (reliable network)...
... Passed -- time 2.8s #peers 3 #RPCs 48 #Ops 0
Test (3B): test progressive failure of followers (reliable network)...
... Passed -- time 5.0s #peers 3 #RPCs 70 #Ops 0
Test (3B): test failure of leaders (reliable network)...
... Passed -- time 5.3s #peers 3 #RPCs 102 #Ops 0
Test (3B): agreement after follower reconnects (reliable network)...
... Passed -- time 6.3s #peers 3 #RPCs 77 #Ops 0
Test (3B): no agreement if too many followers disconnect (reliable network)...
... Passed -- time 4.2s #peers 5 #RPCs 136 #Ops 0
Test (3B): concurrent Start()s (reliable network)...
... Passed -- time 1.0s #peers 3 #RPCs 20 #Ops 0
Test (3B): rejoin of partitioned leader (reliable network)...
... Passed -- time 8.5s #peers 3 #RPCs 125 #Ops 0
Test (3B): leader backs up quickly over incorrect follower logs (reliable network)...
... Passed -- time 32.5s #peers 5 #RPCs 2064 #Ops 0
Test (3B): RPC counts aren't too high (reliable network)...
... Passed -- time 2.0s #peers 3 #RPCs 42 #Ops 0
Test (3C): basic persistence (reliable network)...
... Passed -- time 6.5s #peers 3 #RPCs 75 #Ops 0
Test (3C): more persistence (reliable network)...
... Passed -- time 17.2s #peers 5 #RPCs 298 #Ops 0
Test (3C): partitioned leader and one follower crash, leader restarts (reliable network)...
... Passed -- time 2.6s #peers 3 #RPCs 29 #Ops 0
Test (3C): Figure 8 (reliable network)...
... Passed -- time 33.6s #peers 5 #RPCs 842 #Ops 0
Test (3C): unreliable agreement (unreliable network)...
... Passed -- time 2.2s #peers 5 #RPCs 1073 #Ops 0
Test (3C): Figure 8 (unreliable) (unreliable network)...
... Passed -- time 45.6s #peers 5 #RPCs 10003 #Ops 0
Test (3C): churn (reliable network)...
... Passed -- time 16.3s #peers 5 #RPCs 3464 #Ops 0
Test (3C): unreliable churn (unreliable network)...
... Passed -- time 16.2s #peers 5 #RPCs 2680 #Ops 0
Test (3D): snapshots basic (reliable network)...
... Passed -- time 4.7s #peers 3 #RPCs 502 #Ops 0
Test (3D): install snapshots (disconnect) (reliable network)...
... Passed -- time 45.6s #peers 3 #RPCs 1149 #Ops 0
Test (3D): install snapshots (disconnect) (unreliable network)...
... Passed -- time 84.4s #peers 3 #RPCs 1606 #Ops 0
Test (3D): install snapshots (crash) (reliable network)...
... Passed -- time 36.1s #peers 3 #RPCs 1014 #Ops 0
Test (3D): install snapshots (crash) (unreliable network)...
... Passed -- time 60.1s #peers 3 #RPCs 1325 #Ops 0
Test (3D): crash and restart all servers (unreliable network)...
... Passed -- time 22.3s #peers 3 #RPCs 332 #Ops 0
Test (3D): snapshot initialization after crash (unreliable network)...
... Passed -- time 7.4s #peers 3 #RPCs 102 #Ops 0
PASS
ok      6.5840/raft1    485.208s

```

در نهایت، با اجرای کامل دستور `go test -race` که ترکیبی از تمام تست های پروژه است، موفقیت کامل الگوریتم پیاده سازی شده در برابر تمام سناریوهای پیشنهادی در صورت پروژه تایید شد. تصویر نهایی زیر تایید می کند که در این اجرا نیز تمامی بخش ها PASS شده اند:

مراجع